Bridging Natural Language to Verified Implementation: A Neuro-Symbolic High-Assurance MBSE Pipeline in SysML v2

authors

***Abstract*—The development of high-confidence avionics systems demands rigorous allocation and end-to-end traceability from requirements to implementation, yet current Model-Based Systems Engineering (MBSE) environments provide little support for automated, mathematically verified allocation and traceability. While Generative AI (GenAI) offers a potential automation boon, its deployment in safety-critical workflows is severely constrained by hallucinations, prohibitive token costs, scarcity of open-source domain data, and restrictions on fine-tuning state-of-the-art proprietary models such as GPT-5.**

To address these barriers, we introduce a neuro-symbolic Spec–Code–Proof (SCP) copilot that integrates Codex-class agents into the INSPECTA formal methods pipeline, developed on the DARPA PROVERS program. The pipeline translates natural-language requirements into SysML v2 models containing formal GUMBO contracts, generates HAMR infrastructure code, and discharges Logika proofs, ensuring properties are mathematically preserved in the implementation. To enable this without fine-tuning, we introduce a *Meta-Rule* learning methodology that functions as a verifiable surrogate for weight adaptation. By crystallizing ephemeral in-context learning into persistent, expert-curated abstraction patterns, Meta-Rules drive a self-healing generate–verify–repair loop until verification constraints are formally satisfied within a given budget.

We evaluate our methodology using an example sourced from the FAA Requirements Engineering Management Handbook. Our copilot achieved 100% code-level verification and a 28× increase in development task throughput compared to a baseline that, lacking Meta-Rules, failed to produce verifiable artifacts. This demonstrates a practical path to certification-grade automation under strict data and model governance constraints, reconciling the efficiency of GenAI with the strict rigor required for certifiable critical system software.

***Index Terms*—SWE agents, Neuro-symbolic AI, Formal Methods, SysML v2, MBSE, Supervised Self-Healing, Supervised Meta-Rule Adaptation.**

I. Introduction

High-assurance avionics software development faces a persistent tension: system complexity continues to increase, while certification demands strong evidence that implementation faithfully realizes requirements and architectural intent. In practice, maintaining end-to-end traceability from natural-language requirements to architectural models to executable code remains costly and fragile. In many Model-Based Systems Engineering (MBSE) workflows, requirements allocation and refinement still involve manual, tool-fragmented steps that weaken (or sever) the link between model-level assurance and the software implementation correctness. The resulting drift is often detected late—during integration, verification/validation, or penetration testing—driving rework, schedule risk, and certification friction [1].

Large Language Models (LLMs) appear well-suited to assist with this decomposition by translating requirements into structured specifications, contracts, and implementation scaffolding [2]. However, directly deploying LLMs in safety-critical workflows faces structural barriers. Beyond hallucinations and token-heavy prompting, aerospace environments face the *fine-tuning bottleneck*: domain-representative data is often scarce or proprietary, and fine-tuning frontier proprietary models (e.g., GPT-5-class systems [3]) are typically restricted by vendor policy and/or cost [2], [4].

In this setting, the key challenge therefore is not merely to generate plausible formal text, but rather to produce a *verified* artifact within bounded time, token, and human-review budgets. Recent theory on AI agents in verifiable settings provides a useful lens for this problem: past experience matters to the extent that it captures reusable algorithmic structure that reduces the time needed to solve new tasks [5]. Viewed through this lens, the important quantity is not raw fluency alone, but *cost per verified task*. This framing is particularly natural in MBSE pipelines where a formal checker is available and all candidate artifacts are filtered through hard acceptance gates. These restrictions leave three highly desirable capabilities unmet: (i) reducing token-heavy, brittle prompting; (ii) filtering or rejecting hallucinated behaviors with machine-checkable acceptance gates; and (iii) working from natural language specifications to produce reusable, auditable evidence that persists across runs.

To address these barriers, we introduce a neuro-symbolic **Spec–Code–Proof (SCP)** *copilot*—a generate–verify–repair pipeline that turns natural-language requirements into traceable formal artifacts and verified implementations. SCP targets the three gaps above by (i) shifting effort from repeated token-heavy prompting into reusable intermediate artifacts; (ii) using formal verification as the primary acceptance gate to catch hallucinated or underspecified behavior; and (iii) providing a low-entry natural language interface while accumulating governed, version-controlled specialization knowledge. Our approach integrates Codex-class software engineering agents into the Industrial-Scale Proof Engineering for Critical Trustworthy Applications (**INSPECTA**) toolchain [1], developed on the DARPA Pipelined Reasoning of Verifiers Enabling Robust Systems (PROVERS) program. INSPECTA provides the formal backbone: **SysML v2** [6], [7] captures the system architec-

ture, **GUMBO** [8], [9] expresses behavioral assume/guarantee contracts, **HAMR** [10], [11] generates executable memory-safe infrastructure code, and **Logika** [11] discharges proof obligations using SMT solvers (e.g., Z3 [12] and CVC5 [13]).

A major goal of INSPECTA is to create an auditable workflow where the toolchain mechanically preserves properties established at the model level in the generated code. However, while this infrastructure ensures correctness *after* specification, the initial creation of formal models and contracts remains a manual, expertise-heavy exercise. Ideally, an agentic copilot would assist engineers in synthesizing these rigorous specifications from natural language directly; however, realizing this capability faces the critical barrier of effectively *specializing* an LLM to such a complex, verification-centric toolchain under base-model fine-tuning restrictions.

Recent mechanistic interpretability results suggest that In-Context Learning (ICL) can act like an "implicit fine-tuning" process during inference [14], but the effect is transient: once the context window is cleared, the learned behavior is lost. In high-assurance settings, this ephemerality is a significant obstacle: it forces repeated token-heavy prompting and makes it difficult to govern, audit, or reuse what the model has "learned" across runs.

To stabilize this transient learning into a permanent asset, we introduce *Meta-Rules*: persistent, human-readable formalization abstractions that *crystallize* successful ICL interactions (derived from golden examples, tool feedback, and expert edits) into a reusable rule library. Meta-Rules serve as a verifiable surrogate for black-box weight adaptation: instead of modifying parameters, the system externalizes adaptation into version-controlled artifacts that are (i) inspectable by humans, (ii) exercised by the toolchain, and (iii) constrained by formal verification. In developer mode, the system iteratively proposes and refines Meta-Rules using semantic similarity metrics and INSPECTA tool (e.g., HAMR, Logika) feedback; only candidates that satisfy quantitative improvement criteria, pass verification, and receive human approval are retained. In user mode, the copilot injects this curated library into the context, effectively triggering the underlying implicit fine-tuning mechanism [14] to behave as a domain-adapted agent. This allows us to bypass the aforementioned fine-tuning restrictions while achieving comparable specialization, driving a generate–verify–repair loop until model integration and code-level verification succeed. In addition to consistently producing practical formally verifiable output at low cost (see Section VII), and thus effectively lowering the entry barrier for users, this method systematically accumulates a dataset of verified repair trajectories, alleviating current data scarcity and paving the way for future fine-tuning as model accessibility improves.

By anchoring the development workflow in these governed, externalized artifacts rather than opaque model behaviors, our approach aligns with the industry's emerging *Specification-Driven Development (SDD)* practices. Frameworks such as AWS Kiro [15] aim to curb "vibe coding" by structuring AI-assisted development around natural-language specifications and acceptance criteria rather than unconstrained prompt-driven generation. We extend this paradigm to *Formal Specification-Driven Development (FSDD)* by making *formal verification* the primary acceptance gate. In our workflow, specifications are refined into SysML v2 models and formal GUMBO contracts, then discharged through HAMR/Logika verification. This yields machine-checkable evidence alongside generated code, enabling a workflow aligned with certification demands for traceability, repeatability, as well as auditable assurance artifacts.

In summary, the main contributions of this paper are:

1) **Trustworthy English-to-Implementation Pipeline:** A multi-agent SCP generate–verify–repair loop workflow integrated with INSPECTA (SysML v2 + GUMBO + HAMR/Logika) that generates verified contracts and code from requirements.
2) **Supervised Meta-Rules Self-Adaptation Algorithm:** A verifiable surrogate for fine-tuning that crystallizes ephemeral ICL behaviors into persistent Meta-Rules, enabling a governed generate–verify–repair loop driven by formal verification feedback and human approval.
3) **Benchmark Evaluation:** An evaluation of the Isolette benchmark [16] (a proxy for high-assurance avionics subsystems), demonstrating a substantial increase in verified performance per minute and complete verification success compared to a failing baseline.

The remainder of this paper is structured as follows. After establishing the SysML v2 GUMBO and HAMR/Logika background in Section II-A, Sections III and IV introduce our core methodology: the Spec-Code-Proof architecture, the Meta-Rules framework, and the supervised self-adaptation algorithm. Section V compares Meta-Rules to fine-tuning and transient prompting. Section VI presents the Isolette running example, and Section VII details the experimental evaluation and metrics. Section VIII reviews related work, Section IX discusses limitations and future research, and Section X concludes.

To foster community collaboration and accelerate future research, all toolchain source code, along with our curated dataset of successful repair trajectories, is released as open source.[1]

II. Background

A. INSPECTA High-Assurance System Development and Verification

The development of safety-critical systems, including digital avionics systems, requires rigorous evidence that implementations faithfully realize their requirements. The DARPA PROVERS INSPECTA toolchain provides this verifiable setting by automatically integrating architectural design, code generation, and formal verification capabilities. Specifically,

- SysML v2 and GUMBO: Architecture models are captured in SysML v2, which natively supports a textual representation amenable to LLM processing. These models

[1]URL to be added in the camera-ready version.

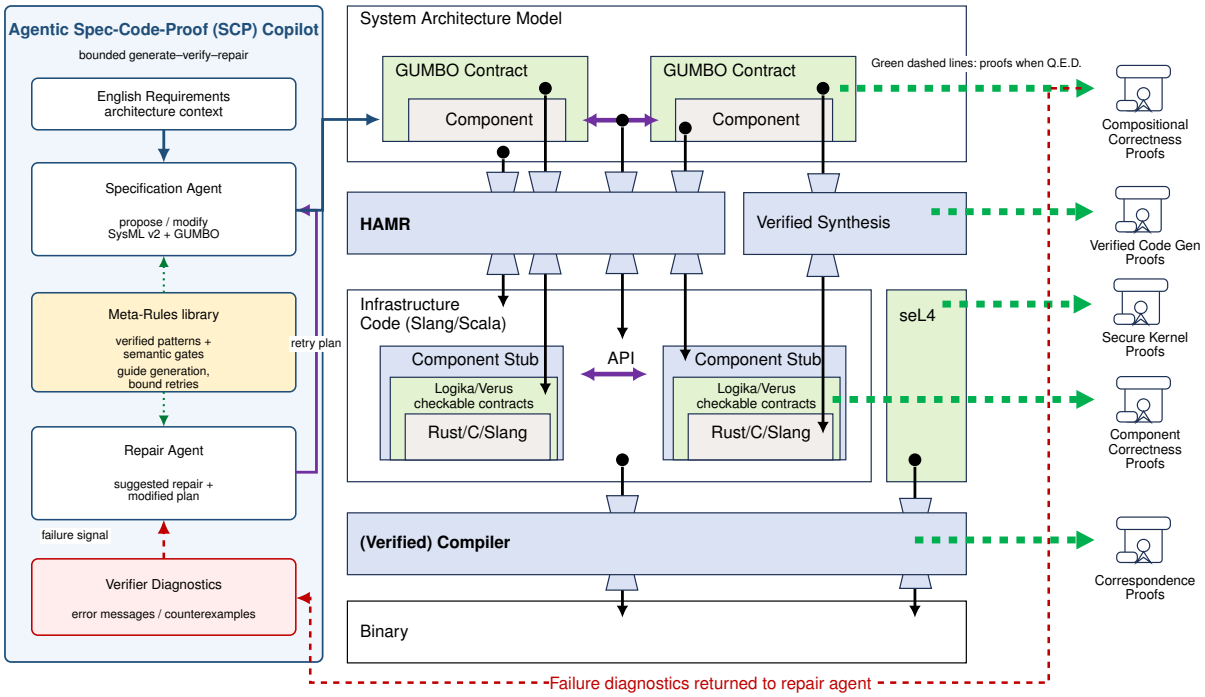

Fig. 1. Agentic SCP/INSPECTA workflow integrating the high-assurance development layers with a Meta-Rule-guided generate–verify–repair loop. Green dashed arrows denote successful proof artifacts; the red dashed arrow denotes verifier-failure diagnostics returned to the repair agent for a bounded retry.

are annotated with GUMBO contracts. GUMBO allows engineers to attach formal specifications in the form of *assumptions* and *guarantees* directly to components (parts) in the architecture model.

- HAMR and Logika: The HAMR framework analyzes these models to generate memory-safe executable infrastructure code in languages like Slang/Scala (default and target of this paper), C, or Rust; mapping GUMBO assumptions and guarantees to formal verifiable *requires* and *ensures* clauses, respectively. Subsequently, in the case of Scala/Slang, the Logika formal verification framework uses SMT solvers (e.g., Z3, CVC5) to discharge proof obligations, ensuring the component application code (developed manually or via LLM assistance) satisfies its GUMBO contracts across the entire state space.
- Finally, Rust and Verus: For Rust targets, HAMR generates Verus verification annotations. Verus allows developers to specify and prove properties using Rust's ownership model, mapping GUMBO assumptions and guarantees to Verus *requires* and *ensures* blocks, respectively. In this paper, our SCP copilot supports the default behavior.

Figure 1 merges the INSPECTA assurance stack with the SCP agentic repair loop used in this paper. The specification agent proposes or updates SysML v2/GUMBO contracts under guidance from persistent Meta-Rules. The verifier stack then acts as a hard gate: successful checks emit proof artifacts, while failed checks return diagnostics and counterexamples to the repair agent. The repair agent converts those diagnostics into a suggested repair and modified plan, which is fed back to the specification agent for a bounded retry. In this sense, Meta-Rules and verifiers act together as guided generation: they constrain the search space before expensive multilayer verification is attempted and prevent unbounded token-heavy trial-and-error.

A typical GUMBO block includes optional state variables, initialization guarantees, assumptions constraining inputs, global guarantees, and `compute_case` clauses that align conditional requirements with case-specific guarantees. Listing 1 provides an illustrative example.

```
language "GUMBO" /* {
  state lastCmd: On_Off;
  initialize
    guarantee initLastCmd: lastCmd == Off;

  compute
    assume lowerLEUpper: lower <= upper;
    guarantee lastCmdTracksOutput: lastCmd ==
        heat_control;

  compute_cases
    case REQ_1:
      assume mode == INIT;
      guarantee heat_control == Off;
    case REQ_2:
      assume mode == NORMAL && temp < lower;
      guarantee heat_control == On;
} */
```

Listing 1. Illustrative skeleton of a GUMBO contract on a SysML v2 component.

III. Neuro-Symbolic Contract Generation with Meta-Rules: An Overview

A. Meta-Rules: Persistent, Auditable Formalization Abstractions

Meta-Rules are persistent, human-readable abstractions that constrain how the copilot translates natural-language requirements and architectural context into GUMBO contracts. Each Meta-Rule captures a reusable mapping from recurring requirement patterns (e.g., hysteresis, "no change" semantics, timeouts, latched modes) to concrete, tool-compatible GUMBO constructs (e.g., state variables, initialization guarantees, and structured `compute_cases`). In contrast to transient prompt-only conditioning, Meta-Rules are stored locally as version-controlled artifacts (a "rule book") that can be reviewed, curated, and audited by domain experts.

Remark 1. An intuitive proof-engineering analogy is helpful. When a proof engineer observes the same sequence of proof moves recurring across solved examples, the natural response is to package that pattern into a reusable lemma. Future proofs then become shorter because the engineer invokes the lemma rather than rediscovering the derivation from scratch. Solved requirement-to-contract episodes play the role of worked proof cases; a candidate Meta-Rule is the reusable lemma-like hypothesis extracted from them; semantic screening estimates where that hypothesis is likely to apply (a forecast model of its practical reusability to produce a practically verifiable specification); and the external verifier stack, together with human review, determines whether the hypothesis is worth retaining for a given budget. The analogy captures the engineering intuition of our method: reusable formalization skill is learned from solved instances, represented symbolically, and validated by an external checker within a given budget.

B. Operational Modes of Use: User Mode vs. Developer Mode

Operationally, we distinguish two modes:

- **User Mode (application).** End users trigger the copilot with a low entry-point prompt that applies an existing, curated Meta-Rule library to translate requirements into candidate GUMBO contracts. The workflow then executes HAMR/Logika verification; failures trigger bounded, tool-guided repair until integration-level and code-level verification succeed.
- **Developer Mode (supervised self-adaptation learning).** Developers iteratively refine the Meta-Rule library itself. Adaptation is supervised and gated: candidate Meta-Rule modifications are treated as hypotheses and retained only if they improve measurable criteria, pass verification, and receive explicit human approval. This yields cumulative improvement without explicit base-model fine-tuning.

1) Geometric Intuition: Practically Verified Semantic Cone: Figure 2 provides intuition for why Meta-Rules act as a verifiable surrogate for fine-tuning in SCP. After injecting a curated Meta-Rule candidate $\mathcal{R}$, verification-aligned performance forms a local maximum over the base-model manifold

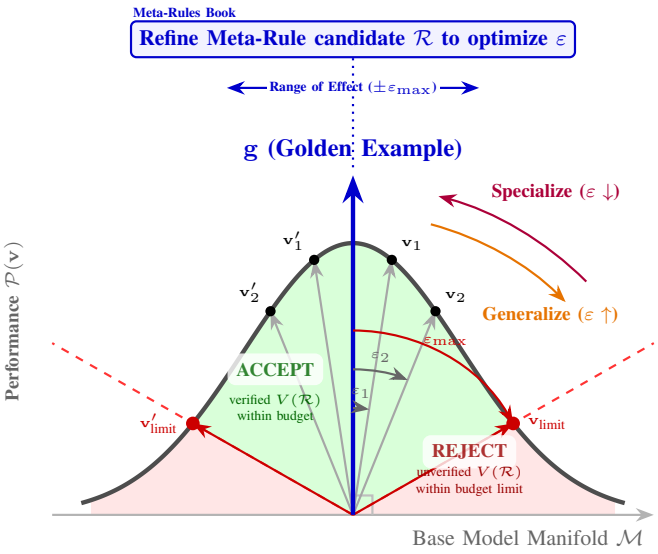

Fig. 2. Practically verified semantic cone for supervised Meta-Rule refinement. A golden example direction $\mathbf{g}$ anchors the accepted region; candidate generations within the cone are retained only when they produce practically verifiable specifications within the available budget, while candidates outside the cone are rejected.

$\mathcal{M}$, centered around a golden example direction $\mathbf{g}$. The Meta-Rule *range of effect* is modeled as an angular neighborhood ($\pm\varepsilon_{\max}$): increasing ε expands the class of requirements and GUMBO contract patterns that can be produced from a small number of exemplars, while remaining inside the accepted budget-verifiable region. If the rule is applied too broadly, repair effort grows and the verifier rejects the candidate. Thus, rule applicability is tuned as an engineering operating point rather than as an unconstrained semantic maximum.

2) Example Meta-Rule: Listing 2 shows an excerpt from the rule book used in our evaluation. The rule instructs the agent to introduce explicit state when the requirements imply memory (e.g., hysteresis bands, "hold previous" semantics, time-dependent predicates). Such rules were motivated by baseline failures in which the agent attempted incompatible idioms (e.g., pre-state operators) and could not consistently realize SysML v2 GUMBO state syntax.

```
/* * 2) WHEN TO USE EACH SECTION
 * 2.1 state
 * Add if the English requires memory:
 * - Hysteresis "shall not be changed" / "hold
     previous"
 * (e.g., Heat within desired band; Alarm no-change
     band).
 * - Timeouts (duration in mode > 1.0 s) if not
     directly
 * available from the platform as a primitive. */
// Pattern examples provided to the Agent:
state
  lastHeat: Isolette_Data_Model::On_Off;
  initElapsed: Base_Types::S64; // time units
```

Listing 2. Extract from GUMBO FSE Agent Plan: Meta-Rules for State Formalization.

IV. Supervised Meta-Rule Self-Adaptation (Learning) and Acceptance Criteria

A. The Algorithm

Algorithm 1 introduces a dual-phase neuro-symbolic process. Let R denote the target set of natural-language requirements, and let $\text{SAMPLE}(R) \subset R$ denote the sampled training subset used in developer mode during Phase I—in our setting, its size was 1/6 of R. Let $G = \{(r_g, g_g)\}$ denote the set of golden examples, where r_g is a golden requirement and g_g is its verified formalized GUMBO contract. Let M denote the retained Meta-Rule library, C the set of verified contracts, and F the set of flagged failures.

Phase I: Meta-Rule extraction, sample-based training, and predictive ϵ-learning. For each golden pair $(r_g, g_g) \in G$, ExtractRule derives a candidate Meta-Rule m. For each sampled requirement $r \in \text{Sample}(R)$, ApplyRule uses m to produce a candidate formalized contract $\hat{g}$. The quantity $\text{Distance}(\hat{g}, g_g)$ denotes a *semantic* distance between the generated formalized contract and the golden formalized contract, rather than a literal textual comparison. The threshold ϵ therefore controls the admissible degree of abstraction away from the golden exemplar. The objective is not to drive ϵ toward zero: if ϵ is too small, the rule becomes overspecialized and loses reusability; if ϵ is too large, the rule admits overly broad generations that require excessive repair or fail to verify within budget B. Accordingly, in developer mode ϵ is tuned across repeated calibration attempts to find a satisfactory intermediate value that best balances rule reuse and bounded repair. Verification is performed by $V(\hat{g}, B)$, where B is the repair budget in time and tokens for full repair cycles; the verifier returns a Boolean result ok together with feedback fb. While $\text{RepairBudgetRemaining}(B)$ holds, $\text{Repair}(\hat{g}, fb, B)$ revises the candidate using verifier feedback. The counter $okCount$ records how many sampled requirements are successfully verified for the candidate rule m, and τ is the minimum acceptance threshold required to retain m in M. If performance is unsatisfactory, DiscardOrRefineWithHuman(m) permits human refinement of the rule, of ϵ, or both, and the developer-mode loop repeats.

Phase II: held-out validation and rule-guided formalization with bounded repair. After Phase I, the retained pair (M, ϵ) is evaluated beyond the sampled training subset. In developer mode, the Phase II loop is instantiated on requirements not used in $\text{Sample}(R)$, testing whether the learned Meta-Rules and calibrated threshold ϵ generalize to unseen requirements while remaining repairable within budget B. In user mode, the same rule-guided synthesis procedure is applied to the full deployment set R. For each requirement r, SynthesizeWithRules(r, M) produces a candidate contract $\hat{g}$, which is checked again by $V(\hat{g}, B)$ and, if needed, revised by $\text{Repair}(\hat{g}, fb, B)$ while budget remains. If verification succeeds, the pair $(r, \hat{g})$ is added to C; otherwise the tuple $(r, \hat{g}, fb)$ is added to F. Operationally, the developer-mode loop (Phase I and Phase II) repeats until a satisfactory ϵ is found, namely one for which the retained Meta-Rule library remains reusable on unseen requirements within the available repair budget.

Algorithm 1: SCP Meta-Rules Learning, Application, and Verification-Guided Repair

Input: Requirements set R; golden examples G (English + formalized GUMBO); verifier V (HAMR/Logika); budgets B (time/tokens/repair cycles); similarity threshold ϵ; acceptance threshold τ.

Output: Verified contracts C; retained Meta-Rules library M; flagged failures F.

Initialization:

$M \leftarrow \emptyset$ `// persistent rule memory`

$C \leftarrow \emptyset$; $F \leftarrow \emptyset$

Phase I: Meta-Rule extraction and validation

foreach $(r_g, g_g) \in G$ **do**
- $m \leftarrow \text{ExtractRule}(r_g, g_g)$
- $okCount \leftarrow 0$
- **foreach** $r \in \text{Sample}(R)$ **do**
  - $\hat{g} \leftarrow \text{ApplyRule}(m, r)$
  - **if** $\text{Distance}(\hat{g}, g_g) > \epsilon$ **then**
    - **continue**
  - $(ok, fb) \leftarrow V(\hat{g}, B)$
  - **while** not ok ***and*** $\text{RepairBudgetRemaining}(B)$ **do**
    - $\hat{g} \leftarrow \text{Repair}(\hat{g}, fb, B)$
    - $(ok, fb) \leftarrow V(\hat{g}, B)$
  - **if** ok **then**
    - $okCount \leftarrow okCount + 1$
- **if** $okCount \geq \tau$ **then**
  - $M \leftarrow M \cup \{m\}$ `// retain only if it verifies and generalizes`
- **else**
  - $\text{DiscardOrRefineWithHuman}(m)$

Phase II: Rule-guided formalization with bounded repair

foreach $r \in R$ **do**
- $\hat{g} \leftarrow \text{SynthesizeWithRules}(r, M)$
- $(ok, fb) \leftarrow V(\hat{g}, B)$
- **while** not ok ***and*** $\text{RepairBudgetRemaining}(B)$ **do**
  - $\hat{g} \leftarrow \text{Repair}(\hat{g}, fb, B)$
  - $(ok, fb) \leftarrow V(\hat{g}, B)$
- **if** ok **then**
  - $C \leftarrow C \cup \{(r, \hat{g})\}$
- **else**
  - $F \leftarrow F \cup \{(r, \hat{g}, fb)\}$

Quality assessment and logging:

Log timestamps, token usage, latency, and repair count; store M for reuse.

B. Practical Considerations

Meta-Rules are stored as local, version-controlled artifacts (a rule book) and are supplied to the agent at execution time together with the verification plan. This design keeps adaptation auditable and allows changes to be reviewed, reproduced, and rolled back. We defer discussion of limitations and trade-offs to Section IX.

C. Discussion: Human-in-the-Loop Refinement

While Meta-Rule extraction and validation are highly automated, human oversight remains essential to ensure transferability and prevent drift. A critical aspect is the **generalization–specialization trade-off**, managed through semantic proximity checks, verification gates, and human governance. The threshold ϵ is not searched for as a theoretical maximum over all admissible semantic deviations. Rather, in developer mode it is calibrated as a user-satisfactory operating point that balances rule reusability against bounded repair cost. Accordingly, SCP adopts a satisficing criterion: ϵ is accepted once the learned pair (M, ϵ) achieves the user-required level of verification performance on unseen requirements within the available budget, rather than when ϵ is theoretically maximal. Human review remains essential in this calibration loop, since the final choice of ϵ reflects an engineering judgment about acceptable trade-offs among generality, verification success, repair effort, and cost.

V. Comparison to Fine-Tuning and Transient Prompting

This section situates Supervised Meta-Rules Adaptation relative to two common approaches for specializing LLMs: parameter fine-tuning and transient prompting / in-context learning (ICL). The comparison emphasizes suitability for high-assurance, certification-oriented workflows where auditability, governance, and reproducibility are first-class requirements.

A. Parameter Fine-Tuning

Fine-tuning modifies model weights to internalize domain-specific behavior. While it can yield strong task performance, it imposes significant practical barriers in aerospace settings: training data are often sensitive or scarce, compute costs can be prohibitive, and distributing tuned checkpoints may be restricted for proprietary frontier models. From an assurance perspective, fine-tuning produces opaque behavioral changes that are difficult to inspect, justify, or certify at the granularity expected in safety-critical development.

B. Transient Prompting and In-Context Learning

ICL adapts behavior via examples and instructions supplied in the prompt window. Mechanistic interpretability work suggests ICL can behave like "implicit fine-tuning" during inference [14], but its effects are ephemeral: the learned behavior disappears when context is reset. In practice, this ephemerality drives repeated long-context prompting, higher token cost, and limited governance over what was learned across sessions.

C. Supervised Meta-Rules Adaptation

Meta-Rules externalize learned formalization behavior into persistent, version-controlled artifacts. Rather than adapting weights, the system crystallizes successful ICL interactions into reusable abstraction patterns that are (i) inspectable by humans, (ii) constrained by formal verification gates (HAMR/Logika), and (iii) retained only under explicit human approval. This transforms adaptation into an auditable engineering process and provides a practical surrogate for fine-tuning under data, cost, and governance constraints.

In this paper, "surrogate for fine-tuning" is meant operationally rather than as a claim of universal equivalence to parameter fine-tuning. Meta-Rules provide persistent, reusable task specialization without modifying model parameters, while preserving auditability, rollback, and verification-gated acceptance. The surrogate claim is therefore about retained specialization behavior under governance constraints, not about matching fine-tuning on every task or benchmark.

Table I summarizes the adaptation mechanisms discussed in this section.

VI. A Running Example: The Isolette System

We use the Isolette Infant Incubator benchmark as a proxy for safety-critical subsystems [16]. The goal is to translate natural-language requirements into SysML v2 GUMBO contracts and then discharge both integration-level and code-level verification obligations using the SCP Copilot Self-healing mechanism with minimal user intervention.

A. Experimental Inputs and Outputs

Inputs. The experiment uses (i) natural-language requirements provided in a realistic FAA-style source document (37+ pages) containing plain text, tables, and figures, and (ii) SysML v2 models and shared libraries representing the Isolette architecture (e.g., `Monitor.sysml`, `Regulate.sysml`, plus SysML standard libraries).These artifacts intentionally include realistic friction: domain terminology, cross-references, and requirements that imply memory (hysteresis/latched state) that must be encoded using SysML v2 GUMBO constructs accepted by the toolchain.

Outputs. The pipeline produces (i) SysML v2 models augmented with GUMBO contracts (including state/initialize/compute_cases blocks), (ii) HAMR-generated implementation artifacts, and (iii) verification logs demonstrating successful discharge of Logika proof obligations for both model integration and code-level execution.

Experimental scope. In this study, the evaluated scope comprised the selected Monitor and Regulate threads, the associated GUMBO contract-bearing blocks and `compute_cases` generated or modified for those threads, and the corresponding HAMR-generated proof-bearing methods examined by Logika (e.g., `initialize` and `timeTriggered`). Accordingly, later references to "complete integration-level and code-level verification" or "100% code-level verification" are with respect to this defined target scope.

```
Baseline: 33 min; no verified artifact.
Parser rejected memory idioms:
  "rejects every variant of the state ... syntax"
  "@pre as an unsupported classification test"
Result: could not encode hysteresis/no-change cases.
Tokens: input=981,770 (+18,673,920 cached),
        output=85,833, reasoning=54,080.
```

Listing 3. Condensed baseline failure excerpt: parser rejection and high token usage.

B. Conditions: Baseline vs. Meta-Rule–Guided SCP

We compare two conditions:

Baseline (prompt-only ICL). A Codex-class agent is tasked to read the requirement document and directly generate correct SysML v2 GUMBO contracts without an external Meta-Rule library. This relies on transient in-context learning and ad-hoc prompt conditioning.

Meta-Rule–guided SCP. The same overall task is executed using a curated Meta-Rule library (Section III). The copilot is triggered with a low entry-point prompt, it applies the rule book to guide formalization, and then executes a bounded generate–verify–repair loop driven by HAMR/Logika feedback. Importantly, the Meta-Rules used in this condition are treated as persistent engineering artifacts: they are created/refined in developer mode and retained only if they satisfy the acceptance gates described in Section III (semantic improvement where golden references exist, verification success, and human approval).

C. Moment 1: Baseline Failure (Token Explosion and Tool Mismatch)

In the initial evaluation, the baseline using Codex GPT-5.1 Max failed to encode requirements that implied memory (hysteresis, "no change") in SysML v2 GUMBO. Listing 3 shows the failure transcript: the agent repeatedly attempts incompatible pre-state idioms (e.g., `@pre`) and cannot converge on tool-accepted GUMBO state syntax. This failure manifests as rapid token growth with no verifiable outcome.

Connection to the comparison. This is a concrete instance of the limitations discussed in Section V: prompt-only ICL is ephemeral and does not reliably internalize niche toolchain syntax, leading to long-context retries and token explosion. A standard remedy would be domain fine-tuning; however, fine-tuning frontier proprietary models was not available in our setting (vendor restrictions). This motivates a persistent, auditable specialization mechanism that does not require weight updates.

```
Meta-Rules: ~25 min; complete success.
Regulate.sysml contracts add state for last mode,
INIT timer, previous actuation, hysteresis, and
REQ_MRM_1--4 / REQ_MHS_1--5 compute cases.
Verification:
  sireum hamr sysml logika --sourcepath isolette/
      sysml
  isolette/hamr/slang/bin/run-logika.cmd
Logika proved all initialise/timeTriggered methods
for Monitor and Regulate target scope.
Tokens: input=317,485 (+4,302,976 cached),
        output=51,833, reasoning=36,032.
```

Listing 4. Condensed Meta-Rule-guided success excerpt: verified target scope.

D. Moment 2: Success via Meta-Rules (Generate–Verify–Repair Under Gates)

Upon applying the curated Meta-Rules (e.g., Listing 2), the SCP copilot successfully formalized the requirements, inserted tool-accepted GUMBO blocks into the model (including state for hysteresis), and completed both integration-level and code-level verification. Listing 4 summarizes the successful run.

Connection to the acceptance gates. The rules used here are not ad-hoc prompts; they are persistent assets curated under the developer-mode gates in Section III. In particular, Meta-Rule updates are retained only when they (i) improve semantic alignment where golden references exist (cosine-distance guidance), (ii) pass integration and code-level verification (hard gate), and (iii) receive human approval prior to being committed to the rule book. The user-mode run then applies these retained rules to reduce search and prevent token-heavy trial-and-error.

VII. Evaluation and Metrics

A. Experimental Settings

The experimental setup follows the scope and two-condition comparison defined in Section VI; this subsection records only the controls needed for metric interpretation. Both runs used the same base-model family, FAA-style requirements source, SysML v2 architecture inputs, shared libraries, and INSPECTA verification workflow. The only intended treatment difference was that the Meta-Rule-guided run received the curated rule library and verification-plan artifacts, whereas the

TABLE I
Qualitative comparison of adaptation mechanisms for high-assurance pipelines.

	Fine-tuning	Prompting/ICL	Meta-Rules
Persistent across runs	Yes	No	Yes (external)
Requires autherization to base model weights updates	Yes	No	No
Cost	High	Med	Low
Auditability / review	Low	Low–Med	High
Governance / rollback	Low	Low	High
Runtime context overhead	Low	High	Low-Med
Data/IP constraints	High	Low	Low
Verification integration	Indirect	Indirect	Direct

baseline relied on prompt-only in-context learning. Success was measured against the target scope defined in Section VI: integration-level and code-level Logika verification over the selected Monitor and Regulate proof-bearing units. Table II reports representative traces under this common setup.

B. Metrics

We report performance using token accounting, wall-clock time, and verification coverage over a defined set of proof-bearing units. Let U denote the set of targeted proof-bearing units within the experimental scope defined in Section VI. In this study, U comprises the selected contract-bearing artifacts and corresponding HAMR-generated proof-bearing methods checked by the INSPECTA workflow.

For a run r, we report wall-clock time $T_{\text{wall}}(r)$, uncached input tokens $N_{\text{in}}(r)$, cached input tokens $N_{\text{cache}}(r)$, reasoning tokens $N_{\text{reason}}(r)$, output tokens $N_{\text{out}}(r)$, and total tokens

$$N_{\text{total}}(r) = \big(N_{\text{in}}(r) + N_{\text{cache}}(r)\big) + N_{\text{out}}(r).$$

Let $U_{\text{verified}}(r) \subseteq U$ denote the subset of targeted units whose obligations are discharged by the end of run r. We then define

$$\rho(r) = \frac{|U_{\text{verified}}(r)|}{|U|},$$

as **verification coverage**,

$$\Theta(r) = \frac{|U_{\text{verified}}(r)|}{T_{\text{wall}}(r)},$$

as **verified throughput** (units/min), and

$$\kappa(r) = \frac{N_{\text{total}}(r)}{|U_{\text{verified}}(r)|},$$

as **token cost per verified unit**. When $|U_{\text{verified}}(r)| = 0$, this quantity is infinite; in our setting, that infinity indicates infeasible cost because no proof-bearing unit was verified despite the consumed tokens.

We also report an end-to-end success indicator $S(r) = 1$ iff $\rho(r) = 1$ and both integration-level and code-level verification complete over the targeted scope; otherwise $S(r) = 0$. Accordingly, "100% code-level verification" in this paper means full coverage of the targeted code-level proof-bearing units within the defined experimental scope.

C. Performance Comparison

The quantitative impact of Meta-Rules is visualized in Fig. 3. The orange bars represent the failing baseline with no Meta-Rules, while the blue bars represent the fine-tuned Meta-Rule–guided workflow.

As illustrated in Fig. 3:

- **Token efficiency:** The baseline consumed total ≈19.75M input tokens and ≈18.6M cached tokens and still failed. With Meta-Rules, input total tokens dropped to ≈4.7M and cached tokens to ≈4.3M, while verification succeeded.
- **Reasoning efficiency:** Reasoning tokens decreased from ≈54k to ≈36k, and output tokens decreased from ≈85k to ≈51.8k, consistent with reduced search and fewer failed retries.
- **Wall-clock time:** Total runtime decreased from 33 minutes for the largely failing baseline to ≈ 25 minutes for the fully successful Meta-Rule-guided run.
- **Verified throughput (Θ):** Over the targeted proof-bearing units defined above, the Meta-Rule–guided approach achieved a reported $\approx 28\times$ improvement in verified throughput, i.e., $\Theta_{\text{MR}}/\Theta_{\text{Baseline}} \approx 28$, while also reaching end-to-end success.
- **Token cost per verified unit (κ):** Meta-Rules reduced κ from ≈ 9.87M to ≈ 111K tokens per verified unit, a nearly two-order-of-magnitude reduction ($\approx 89\times$ lower, or about 1.1% of the baseline cost).
  Table II reports the corresponding scope and metric fields ($|U|$, $|U_{\text{verified}}|$, ρ, Θ, and κ) for both conditions.

D. Representative Results

Table II reports representative logs from the baseline and Meta-Rule-guided runs. The baseline failed to produce a verifier-accepted artifact despite substantially higher token expenditure, whereas the Meta-Rule-guided run completed both integration-level and code-level verification.

The quantitative impact of Meta-Rules is also visualized in Fig. 3. The orange bars represent the failing baseline with no Meta-Rules, while the blue bars represent the fine-tuned Meta-Rule-guided workflow.

Several observations follow from Table II. First, Meta-Rules convert a largely failing baseline run into one that completes formal verification. Second, cached input tokens decrease substantially, from ≈ 18.67M to ≈ 4.30M, reducing context-replay overhead and preserving more of the execution budget for productive verification and repair. Third, reasoning tokens decrease from 54,080 to 36,032, consistent with the claim that the rule-guided workflow reduces the most expensive unproductive token-reasoning cost. Fourth, wall-clock time decreases from 33 minutes to about 25 minutes while achieving complete verification and improving verified throughput, Θ, from 0.06 to 1.68 units/min, an approximately $28\times$ increase. Finally, the successful run reduces the token cost per verified unit, κ, by about $89\times$, from ≈ 9.87M to ≈ 111K tokens per verified unit, or to approximately 1.1% of the baseline cost. Taken together, these results show that reduced search cost is accompanied by a meaningful runtime gain, rather than only by improved final correctness.

VIII. Related Work

LLMs have increasingly been used to assist formal reasoning and proof engineering. Early work such as GPT-f explored generative theorem proving in Metamath [18], while later systems studied whole-proof generation, proof repair, and diagnosis in proof assistants such as Isabelle/HOL and Coq [19], [20]. In code verification, neuro-symbolic workflows have also appeared around F* [21], SMT-backed hallucination checks [22], and symbolic reasoning benchmarks such as GSM-Symbolic [23].

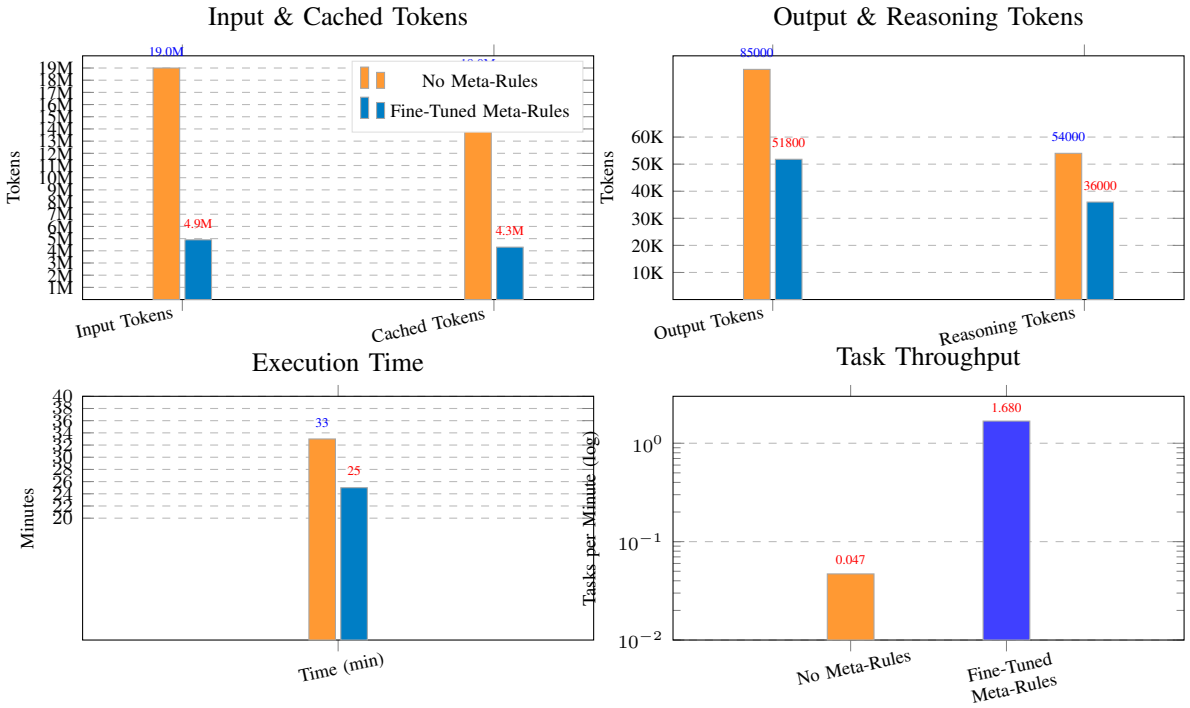

Fig. 3. Performance comparison between the prompt-only baseline and the Meta-Rule-guided SCP workflow. The charts report token use, wall-clock time, and verified throughput from the representative traces in Table II.

TABLE II
REPRESENTATIVE RUN METRICS FOR THE BASELINE AND META-RULE-GUIDED SCP CONDITIONS.

Metric	Baseline (prompt-only)	Meta-Rule-guided SCP
T_{wall} (minutes)	33	25
N_{in}	981,770	317,485
N_{cache}	18,673,920	4,302,976
N_{out}	85,833	51,833
N_{reason}	54,080	36,032
N_{total}	19,741,523	4,672,294
$\lvert U \rvert$ (targeted proof-bearing units)	*42*	*42*
$\lvert U_{\text{verified}} \rvert$ (verified units)	*2*	*42*
ρ (verification coverage)	4.76%	100%
Θ (verified throughput, units/min)	*0.06*	*1.68*
κ (token cost per verified unit)	$\approx 9.87M$	$\approx 111K$

Formal methods are also being integrated into MBSE standards and tools. SysML v2 was designed with formal semantics in mind [24], and Imandra has demonstrated model-level SysML v2 verification [25]. Closer to our architecture, AGREE-Dog introduced a neuro-symbolic copilot for AADL/AGREE model repair [26], [27]. Our contribution differs by integrating the INSPECTA SCP path from natural language requirements to GUMBO contracts, HAMR-generated code, and Logika proof discharge, thereby targeting end-to-end verified implementation rather than only model-level analysis.

IX. Limitations and Future Work

This evaluation is an initial study over the Isolette target scope, not a claim of universal generalization. Future work will expand evaluation to heterogeneous cyber-physical and security components, including seL4-based firewall architectures [28]; extend the self-healing loop from Scala/Slang to Rust/Verus application code using repair heuristics inspired by VERISTRUCT [29]; and scale verification through cloud execution, multimodal physically-aware specification, and compositional reasoning advances in GUMBO/AGREE [26], [30]. Further details will be provided in an extended version of this paper[2].

X. Conclusion

This paper shows that persistent, verifier-gated Meta-Rules can turn transient LLM formalization behavior into reusable engineering knowledge for high-assurance MBSE. Integrated with INSPECTA, the SCP copilot converts natural language

[2]To be made available following review process.

requirements into GUMBO contracts for SysML v2 models, HAMR-generated code, and Logika-verified artifacts. On the Isolette target example, Meta-Rules converted a failing prompt-only run into a verified run while reducing token and time costs, supporting a practical path towards auditable, certification-ready automation.

ACKNOWLEDGMENT

This work was funded by DARPA. The views, opinions, and/or findings expressed are those of the authors and should not be interpreted as representing the official views or policies of DARPA, the U.S. Department of Defense, or the U.S. Government.

REFERENCES

[1] D. Hardin, I. Amundson, J. Babar, D. Cofer, S. Hasan, K. Hoech, J. Belt, J. Hatcliff, Robby, and S. Hallerstede, "Automated SysML v2 system model to memory-safe language code generation for avionics applications," in *Proceedings of the IEEE/AIAA Digital Avionics Systems Conference (DASC)*, 2025, author version / preprint (uploaded as hardin2025dasc.pdf).

[2] OpenAI, "GPT-5.2 Codex System Card," https://openai.com/index/gpt-5-2-codex-system-card/, 2025, accessed: 2026-02-12.

[3] S. Bubeck, C. Coester, R. Eldan, T. Gowers, Y. T. Lee, A. Lupsasca, M. Sawhney, R. Scherrer, M. Sellke, B. K. Spears, D. Unutmaz, K. Weil, S. Yin, and N. Zhivotovskiy, "Early science acceleration experiments with GPT-5," 2025. [Online]. Available: https://arxiv.org/abs/2511.16072

[4] OpenAI, "GPT-5 System Card," https://openai.com/index/gpt-5-system-card/, 2025, accessed: 2026-02-12.

[5] A. Achille and S. Soatto, "AI agents as universal task solvers," *Entropy*, vol. 28, no. 3, 2026. [Online]. Available: https://www.mdpi.com/1099-4300/28/3/332

[6] Object Management Group (OMG), "SysML v2 model formalism working group," https://www.omgwiki.org/OMGSysML/doku.php?id=sysml-roadmap:sysml_v2_model_formalism_working_group, accessed: 2026-02-10.

[7] Object Management Group, "OMG system modeling language (SysML) v2 specification," 2025. [Online]. Available: https://www.omg.org/spec/SYSML

[8] J. Hatcliff, D. Stewart, J. Belt, Robby, and A. Schwerdfeger, "An AADL contract language supporting integrated model- and code-level verification," *Ada Letters*, vol. 42, no. 2, p. 45–54, April 2023. [Online]. Available: https://doi.org/10.1145/3591335.3591339

[9] Galois, "Gumbo: Grand unified modeling of behavioral operators," 2024. [Online]. Available: https://www.galois.com/project/gumbo

[10] J. Hatcliff, J. Belt, Robby, and T. Carpenter, "HAMR: An AADL multi-platform code generation toolset," in *10th International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (ISoLA)*, ser. LNCS, vol. 13036, 2021, pp. 274–295.

[11] *Sireum HAMR: High Assurance Modeling and Rapid Engineering for Embedded Systems*, Kansas State University, 2025. [Online]. Available: http://hamr.sireum.org/

[12] L. De Moura and N. Bjørner, "Z3: An Efficient SMT Solver," in *Proceedings of the Theory and practice of software, 14th international conference on Tools and algorithms for the construction and analysis of systems*, ser. TACAS'08/ETAPS'08, 2008, pp. 337–340.

[13] C. Barrett, C. L. Conway, M. Deters, L. Hadarean, D. Jovanović, T. King, A. Reynolds, and C. Tinelli, "CVC4," in *Proceedings of the 23rd international conference on Computer aided verification*, ser. CAV'11, 2011, pp. 171–177.

[14] J. von Oswald, E. Niklasson, E. Randazzo, J. Sacramento, A. Mordvintsev, A. Zhmoginov, and M. Vladymyrov, "Transformers learn in-context by gradient descent," in *Proceedings of the 40th International Conference on Machine Learning*. PMLR, 2023, pp. 35 151–35 174.

[15] AWS News Blog, "Prevent factual errors from llm hallucinations with mathematically sound automated reasoning checks (preview)," AWS Blog Post, 2024, accessed: 2025-05-11.

[16] J. Hatcliff *et al.*, "The Isolette system: Illustrating end-to-end artifacts for rigorous model-based engineering," in *International Symposium On Leveraging Applications of Formal Methods, Verification and Validation (ISoLA)*, 2024, author version (uploaded as Hatcliff-etal-ISOLA2024-Isolette-Overview.pdf).

[17] S. Polu and I. Sutskever, "Generative language modeling for automated theorem proving," *arXiv preprint arXiv:2009.03393*, 2020.

[18] E. First, M. N. Rabe, T. Ringer, and Y. Brun, "Baldur: Whole-proof generation and repair with large language models," *arXiv preprint arXiv:2303.04910*, 2023.

[19] A. Tahat, D. Hardin, A. Petz, and P. Alexander, "Proof repair utilizing large language models: A case study on the copland remote attestation proofbase," in *Proceedings of International Symposium On Leveraging Applications of Formal Methods Verification and Validation (AISolA)*, 2024.

[20] N. Swamy, C. Hritcu, C. Keller, A. Rastogi, A. Delignat-Lavaud, S. Forest, K. Bhargavan, C. Fournet, P. Strub, M. Kohlweiss, J. K. Zinzindohoue, and S. Z. Béguelin, "Dependent types and multi-monadic effects in F," in *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, R. Bodík and R. Majumdar, Eds. ACM, 2016, pp. 256–270. [Online]. Available: https://doi.org/10.1145/2837614.2837655

[21] Amazon Web Services, "Automated reasoning checks in amazon bedrock guardrails," 2025. [Online]. Available: https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails-automated-reasoning-checks.html

[22] S. I. Mirzadeh, K. Alizadeh, H. Shahrokhi, O. Tuzel, S. Bengio, and M. Farajtabar, "GSM-symbolic: Understanding the limitations of mathematical reasoning in large language models," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=AjXkRZIvjB

[23] Object Management Group (OMG), "Systems modeling language (SysML) v2 specification," , 2024, accessed: 2025-05-19.

[24] Imandra, "Imandra SysML: Formal verification for SysML v2 models," 2024. [Online]. Available: https://www.imandra.ai/sysml

[25] A. Tahat, I. Amundson, D. Hardin, and D. Cofer, "AGREE-Dog copilot: A neuro-symbolic approach to enhanced model-based systems engineering," in *Bridging the Gap Between AI and Reality (AISoLA 2025), Selected Papers*, 2025. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-032-07132-3_8

[26] P. H. Feiler and D. P. Gluch, *Model-Based Engineering with AADL: An Introduction to the SAE Architecture Analysis & Design Language*, 1st ed. Addison-Wesley Professional, 2012.

[27] J. Hatcliff, "Model-based development for seL4 Microkit/Rust with integrated formal methods using HAMR," Presentation at the seL4 Summit 2025, 2025, accessed: 2025-05-19. [Online]. Available: https://sel4summit2025.sched.com/event/26GD3

[28] C. Sun, Y. Sun, D. Amrollahi, E. Zhang, S. Lahiri, S. Lu, D. Dill, and C. Barrett, "VeriStruct: AI-assisted automated verification of data-structure modules in verus," 2025. [Online]. Available: https://arxiv.org/abs/2510.25015

[29] D. Cofer, A. Gacek, S. Miller, M. W. Whalen, B. LaValley, and L. Sha, "Compositional verification of architectural models," in *NASA Formal Methods*, A. E. Goodloe and S. Person, Eds. Springer, 2012, pp. 126–140.